

Lab3 : EEG classification

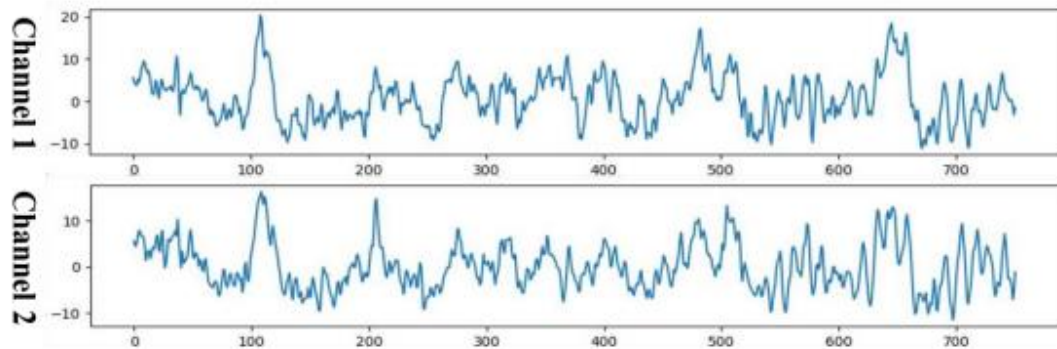
1. Introduction

使用 EEGNet 與 DeepConvNet 來分類 BCI competition dataset，並

且設定 3 個不同的 activation function (ReLU, Leaky ReLU, ELU)、

epoch = 300、learning rate = 0.001、Batch Size = 64 等參數來訓練

模型。下方為 Prepare Data，Train 與 Test Dataset。



2. Experiment set up

A. The detail of your model

```
EEGNet(  
  (firstconv): Sequential(  
    (0): Conv2d(1, 16, kernel_size=(1, 51), stride=(1, 1), padding=(0, 25), bias=False)  
    (1): BatchNorm2d(16, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
  )  
  (depthwiseConv): Sequential(  
    (0): Conv2d(16, 32, kernel_size=(2, 1), stride=(1, 1), groups=16, bias=False)  
    (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
    (2): ELU(alpha=1.0)  
    (3): AvgPool2d(kernel_size=(1, 4), stride=(1, 4), padding=0)  
    (4): Dropout(p=0.25)  
  )  
  (separableConv): Sequential(  
    (0): Conv2d(32, 32, kernel_size=(1, 15), stride=(1, 1), padding=(0, 7), bias=False)  
    (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
    (2): ELU(alpha=1.0)  
    (3): AvgPool2d(kernel_size=(1, 8), stride=(1, 8), padding=0)  
    (4): Dropout(p=0.25)  
  )  
  (classify): Sequential(  
    (0): Linear(in_features=736, out_features=2, bias=True)  
  )  
)
```

```

class EEG(nn.Module):
    def __init__(self, activation = nn.ELU(alpha = 1.0)):
        super(EEG, self).__init__()
        # network layer
        self.firstconv = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=(1, 51), stride=(1, 1), padding=(0, 25), bias=False),
            nn.BatchNorm2d(16, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        )
        self.depthwiseconv = nn.Sequential(
            nn.Conv2d(16, 32, kernel_size=(2, 1), stride=(1, 1), groups=16, bias=False),
            nn.BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True),
            activation,
            nn.AvgPool2d(kernel_size=(1, 4), stride=(1, 4), padding=0),
            nn.Dropout(p=0.25)
        )
        self.separableconv = nn.Sequential(
            nn.Conv2d(32, 32, kernel_size=(1, 15), stride=(1, 1), padding=(0, 7), bias=False),
            nn.BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True),
            activation,
            nn.AvgPool2d(kernel_size=(1, 8), stride=(1, 8), padding=0),
            nn.Dropout(p=0.25)
        )
        self.classify = nn.Sequential(
            nn.Linear(in_features=736, out_features=2, bias=True)
        )

    def forward(self, x):
        # forward propagation
        x = self.firstconv(x)
        x = self.depthwiseconv(x)
        x = self.separableconv(x)
        x = x.view(x.size(0), -1)
        x = self.classify(x)
        # output = F.log_softmax(x, dim=1)
        return x

```

利用上方圖片的資訊，來建立下方的 EEG 的生成網路，其中有使用到幾個參數。

首先是 Conv2d，因為他要做二維捲積的所以用 Conv2d，此為卷積層，其中舉下圖為例：

```
nn.Conv2d(1, 16, kernel_size=(1, 51), stride=(1, 1), padding=(0, 25), bias=False),
```

第一個參數 1 為 **in_channels**，表輸入的圖像通道數

第二個參數 16 為 **out_channels**，表輸出的圖像通道數

第三個參數 **kernel_size**，表捲積的核尺寸，此項為核為(1,51)

第四個參數 **stride**，表捲積的步長度，也就是每次該平移的量。

第五個參數 **padding**，預設為 0，此處(0,25)代表水平方向上的左右邊緣各填充 25 個零值，而在垂直方向上不進行填充

第六個參數 **bias**，表輸出實添加偏差，但是我們不需要所以設為 false。

接著是 BatchNorm2d，他主要是在每一層都做一次正規化，用來減緩梯度消失且可以加速收斂，而其中舉下圖為例：

```
nn.BatchNorm2d(16, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
```

第一個參數 16 為 **num_feature**，表輸入 BN 的通道數

第二個參數 **eps**，在分母多加一個數避免為 0

第三個參數 **momentum**，0~1 的值，接近 1，統計值會更考慮之前的值，更新的速度較慢。而當 **momentum** 的值接近于 0 時，統計值由批次的均值和方差決定。

第四個參數 **affine**，true 時會給定可學習的 **gamma** 和 **beta**

第五個參數 **track_running_stats**，true 時使用的是累積的均值和方差，而不是僅基於當前批次計算的均值和方差。

接著是 **activation**，用來導入些非線性的性質，使 **nn** 可以學習跟表達更複雜的函數，如果沒他每一層就是在做矩陣互乘而已。

再來是 **AvgPool2d**，池化層主要的工作是將資料壓縮，把不必要的空白去掉，把不重要的降躁，增加穩定性，而舉下圖為例:

```
nn.AvgPool2d(kernel_size=(1, 4), stride=(1, 4), padding=0),
```

第一個參數 **kernel_size**，表池化核尺寸，此項為核為(1,4)

第二個參數 **stride**，移動步幅，默認跟 **kernel_size** 一樣。

第三個參數 **padding**，兩側填充的大小。

接著在 **forward function** 那邊就會利用我們剛剛建好的 **EEG Model**，把 **input** 一路 **forward** 到輸出，並且因為我們的 **loss function** 就是利用 **cross entropy**，所以最後就沒有特別再去做 **softmax**。

Layer	# filters	size	# params	Activation	Options
Input		(C, T)			
Reshape		(1, C, T)			
Conv2D	25	(1, 5)	150	Linear	mode = valid, max norm = 2
Conv2D	25	(C, 1)	25 * 25 * C + 25	Linear	mode = valid, max norm = 2
BatchNorm			2 * 25		epsilon = 1e-05, momentum = 0.1
Activation				ELU	
MaxPool2D		(1, 2)			
Dropout					p = 0.5
Conv2D	50	(1, 5)	25 * 50 * C + 50	Linear	mode = valid, max norm = 2
BatchNorm			2 * 50		epsilon = 1e-05, momentum = 0.1
Activation				ELU	
MaxPool2D		(1, 2)			
Dropout					p = 0.5
Conv2D	100	(1, 5)	50 * 100 * C + 100	Linear	mode = valid, max norm = 2
BatchNorm			2 * 100		epsilon = 1e-05, momentum = 0.1
Activation				ELU	
MaxPool2D		(1, 2)			
Dropout					p = 0.5
Conv2D	200	(1, 5)	100 * 200 * C + 200	Linear	mode = valid, max norm = 2
BatchNorm			2 * 200		epsilon = 1e-05, momentum = 0.1
Activation				ELU	
MaxPool2D		(1, 2)			
Dropout					p = 0.5
Flatten					
Dense	N			softmax	max norm = 0.5

```

class DeepConvNet(nn.Module):
    def __init__(self, activation = nn.ELU(alpha = 1.0)):
        super(DeepConvNet, self).__init__()

        # network layer
        self.layer0 = nn.Conv2d(1, 25, kernel_size=(1, 5))

        self.layer1 = nn.Sequential(
            nn.Conv2d(25, 25, kernel_size=(2, 1)),
            nn.BatchNorm2d(25),
            activation,
            nn.MaxPool2d(kernel_size=(1, 2)),
            nn.Dropout(p=0.5)
        )

        self.layer2 = nn.Sequential(
            nn.Conv2d(25, 50, kernel_size=(1, 5)),
            nn.BatchNorm2d(50),
            activation,
            nn.MaxPool2d(kernel_size=(1, 2)),
            nn.Dropout(p=0.5)
        )

        self.layer3 = nn.Sequential(
            nn.Conv2d(50, 100, kernel_size=(1, 5)),
            nn.BatchNorm2d(100),
            activation,
            nn.MaxPool2d(kernel_size=(1, 2)),
            nn.Dropout(p=0.5)
        )

        self.layer4 = nn.Sequential(
            nn.Conv2d(100, 200, kernel_size=(1, 5)),
            nn.BatchNorm2d(200),
            activation,
            nn.MaxPool2d(kernel_size=(1, 2)),
            nn.Dropout(p=0.5)
        )

        self.classify = nn.Sequential(
            nn.Linear(8600, 2)
        )

```

```

def forward(self, x):
    # forward propagation
    x = self.layer0(x)
    x = self.layer1(x)
    x = self.layer2(x)
    x = self.layer3(x)
    x = self.layer4(x)
    x = x.view(x.size(0), -1)
    # print(x.shape)
    x = self.classify(x)
    # output = F.log_softmax(x, dim=1)
    return x

```

利用上方所給的資訊建立 DeepConvNet，總共利用六層來建立此模型，而每個模型內的參數介紹與前述一樣，比較值得注意的是在最後 linear

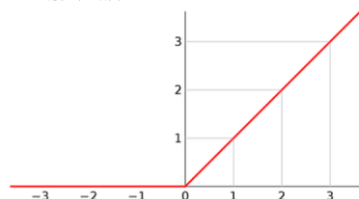
layer 中的輸入是 8600 是因來自於上一層的輸出，最終也跟前者一樣我們的 loss function 是利用 cross entropy，所以最後就不用特別再去做 softmax。

B. Explain the activation function (ReLU, Leaky ReLU, ELU)

(1) ReLU

$$f(x) = \max(0, x)$$

ReLU圖形如下圖所示：



ReLU 將負數輸入值設為零，將正數輸入值保持不變。表達式微上方圖片中的內容。ReLU 的優點是計算效率高，因為它僅涉及簡單的比較操作，並且不需要指數運算而，而 ReLU 的缺點是在負輸入時，導數為零，這意味著相應的神經元將無法進行反向傳播學習。

(2) Leaky ReLU



Leaky ReLU 是解決 ReLU 負輸入時的問題。在負數輸入值上引入一個小的斜率，而不是將其設為零。表達式上方圖片中的內容，其中 0.1 是一個很小的正數。這樣做的好處是，在負輸入時，神經元仍然具有一個小的梯度，從而允許相應的參數進行一些微小的更新。這有助於避免神經元“死亡”的問題，即在訓練過程中永遠不會被激活。

(3) ELU

$$x < 0 : ELU(x) = e^x - 1$$

$$x \geq 0 : ELU(x) = x$$

其函数图像如下图所示

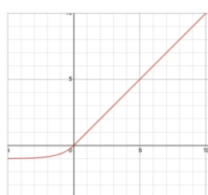


Fig 1. Exponential Linear Unit or ELU activation function

ELU 是解決 ReLU 在負數輸入時的問題，在負數那邊加入非零斜率，

同時保持正數輸入不變。表達是在上方圖片中，且與 ReLU 相比，使用 ELU 可在神經網路中縮短訓練時間並提高準確度。

3. Experimental results

(1) The highest testing accuracy

EEG Net

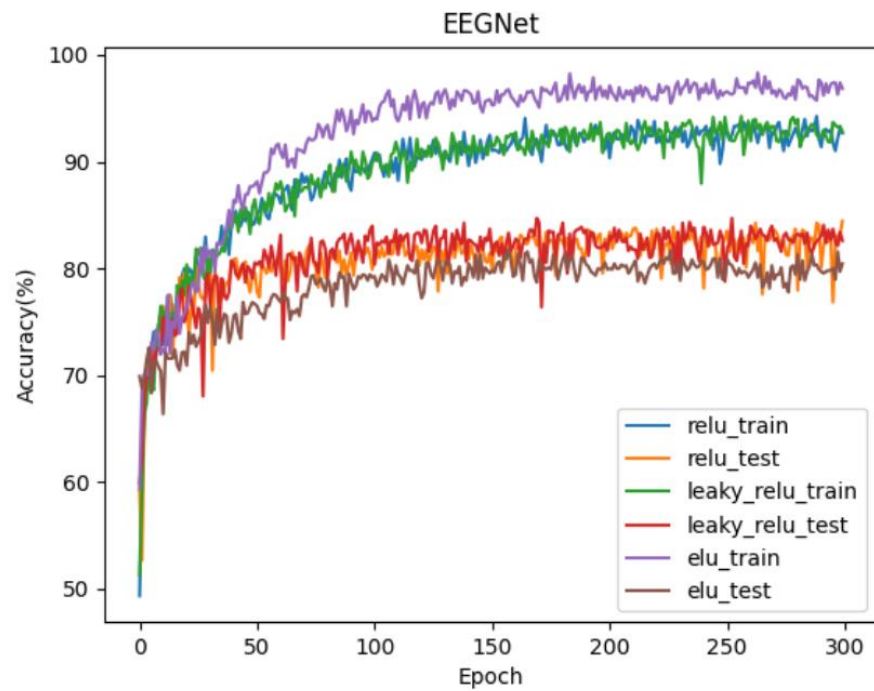
```
EEG Net
relu_train: 98.43%
relu_test: 87.59%
leaky_relu_train: 97.78%
leaky_relu_test: 87.04%
elu_train: 92.87%
elu_test: 85.28%
```

DeepConvNet

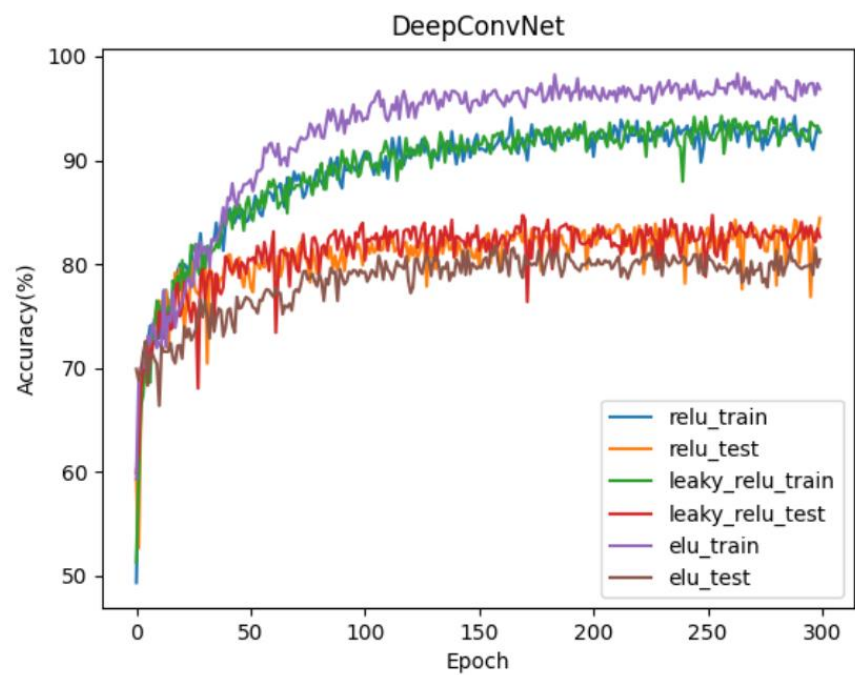
```
DeepConvNet
relu_train: 94.26%
relu_test: 84.44%
leaky_relu_train: 94.26%
leaky_relu_test: 84.72%
elu_train: 98.33%
elu_test: 81.85%
```

(2) Comparison figures

EEG Net



DeepConvNet



4. Discussion

1. 如果不使用 **activation function**，跑出來的效果明顯差很多，
利用 **EEG NET** 舉例，左邊是沒有用激活函數，右邊則有。

<pre>ReLU_train Accuracy: 79.25925925925927% ReLU_test Accuracy: 71.94444444444444% LeakyReLU_train Accuracy: 79.16666666666666% LeakyReLU_test Accuracy: 72.03703703703704% ELU_train Accuracy: 80.27777777777779% ELU_test Accuracy: 71.66666666666667%</pre>	<pre>EEG Net relu_train: 98.43% relu_test: 87.59% leaky_relu_train: 97.78% leaky_relu_test: 87.04% elu_train: 92.87% elu_test: 85.28%</pre>
---	---

2. 在我們將資料集放入 **GPU** 時，我們需要將 **x,y** 都 **to device**，
這時我們需要指定他的 **dtype**，比較特別的是我們的 **y** 需要將
他設置為 **torch.long**，不然會出錯。
3. **Batch Size** 與 **memory** 大小息息相關，而我自己是將它設置到
64 就會是極限了(我用自己的電腦)，而 **Batch size** 越大會讓我們
們的訓練速度越快。
4. 我們在訓練時，會加入一個 **dropout**，他跟 **BatchNorm2d**，都
是在做正規化的事情，但是前者是隨機丟棄一些神經元，來
讓我們的機器避免 **overfitting** 的問題。後面的則是對每個小
batch 做正規化，加速訓練過程，並且增加模型泛化能力。